

Ingeniería Agéntica

Curso de extensión — FaMAF, Universidad Nacional de Córdoba. Docentes: Diego Piloni y Agustín Carrasco. Invitado: Alejandro Silva (CCAD - UNC). Destinatarios: programadores/as Junior y estudiantes avanzados de carreras de computación (cupó de 30 personas). Modalidad presencial teórico-práctica: 6 encuentros semanales, de ~2hs c/u. Cada estudiante trabaja sobre un proyecto propio que comenzarán en el curso y que evoluciona a lo largo de las seis clases.

Requisitos: Notebook, uso de la terminal y git. Se requiere asistir al curso con una suscripción paga de un Lab de IA, de ~20 USD mensuales. Sugeridos: Plan Lite de GLM Coding Plan <https://z.ai/subscribe> de 18 USD mensuales o Plan Plus de Codex <https://chatgpt.com/codex/pricing/> de 20 USD mensuales. (No sirve un plan de Claude dado que solo puede ser ejecutado en Claude Code y en el curso usaremos Pi <https://pi.dev/>, un Harness Open Source)

1. Objetivos

Que los estudiantes recorran el espectro del desarrollo asistido por IA —del *vibe coding* a la ingeniería agéntica— y adquieran el criterio para usar agentes de código de manera responsable, entendiendo que la responsabilidad por el software producido es siempre de la persona y no de la herramienta. Al finalizar, se espera que puedan:

1. Explicar el funcionamiento práctico de un modelo de lenguaje: tokens, ventana de contexto, predicción del siguiente token, alucinaciones.
2. Reconocer los costos ocultos del desarrollo sin estructura: deuda de comprensión, errores en cascada, ilusión de productividad.
3. Planificar antes de ejecutar y revisar críticamente el código generado.
4. Configurar y extender un agente: instrucciones persistentes, habilidades reutilizables (Skills), herramientas externas (MCP).
5. Practicar ingeniería de contexto: distinguir problema de solución, mantener documentación viva y encadenar documento → plan → implementación → revisión.
6. Describir la arquitectura interna de un *harness*: ciclo del agente, extensiones, gestión de contexto y permisos.
7. Aprender sobre modelos de pesos abiertos, leer sus licencias, estimar el hardware necesario y apuntar el mismo agente a otro modelo.
8. Conocer la infraestructura de cómputo de alto desempeño de la UNC (CCAD).

2. Contenidos

Seis unidades, una por encuentro. Las unidades 1 a 4 forman el **módulo base** —¿cómo trabajo con esta herramienta?—, y cada una agrega una capa de estructura como respuesta a un problema de las clases anteriores. Las unidades 5 y 6 forman el **módulo avanzado**, que cambia la pregunta: ¿de qué está hecha y qué pasa si le cambio las partes?

Unidad 1 — La experiencia del *vibe coding*. IA generativa y modelos de lenguaje: predicción del siguiente token, alucinaciones, tokens, multimodalidad, precios. Ventana de contexto como memoria de trabajo finita e higiene de contexto. Chat vs Agents. Anatomía de un agente: modelo + herramientas + *harness*. Vibe coding: definición, origen y crítica —deuda de comprensión e ilusión de productividad—. Práctica: instalación, proyecto propio y sesión de *vibe coding* con reglas estrictas; luego revisión del código producido.

Unidad 2 — Planificación y revisión. El desplazamiento del cuello de botella de escribir a verificar. Uso de Git y revisión de diffs. Planificación: modo plan basado en archivos, iteración con anotaciones. Descomposición de tareas.

Unidad 3 — Herramientas y habilidades. Herramientas (Skills) como unidad de capacidad del agente. Instrucciones persistentes (AGENTS.md). Habilidades y comandos reutilizables según el estándar abierto Agent Skills.

Unidad 4 — Ingeniería de contexto. Dominio del problema y dominio de la solución. Documentación del proyecto como contexto, versionada en el repositorio. Escribir documentación con el agente: el agente pregunta, la persona decide. La cadena documento → plan → implementación → revisión. Cierre del módulo base: deuda cognitiva, costo y límites.

Unidad 5 — Internals de un *harness* de codificación. Demo de los proyectos de los estudiantes. Teoría: El ciclo del agente paso a paso y sus puntos de extensión. Arquitectura por capas. Sesiones como árbol. Tres formas de darle una herramienta a un modelo: extensión, CLI, MCP. Compactación de contexto. Modos de ejecución. Práctica: escribir una extensión propia.

Unidad 6 — Modelos de pesos abiertos y CCAD. Modelos de pesos abiertos. Licencias. Qué hace falta para ejecutar un modelo: formatos, cuantización y el costo de la ventana de contexto en memoria. El endpoint compatible con OpenAI como historia de interoperabilidad; Demostración de un modelo sobre una GPU portátil. Charla de Alejandro Silva, invitado del CCAD: qué es la computación de alto desempeño, el equipamiento de la UNC, el despliegue de modelos como servicio multiusuario y cómo solicitar una cuenta. Práctica: apuntar el agente propio a los modelos del CCAD. Cierre: retrospectiva del curso.

3. Bibliografía

- 3Blue1Brown: Sanderson, G. *Breve explicación de los modelos extensos de lenguaje (LLM)*. <https://www.youtube.com/watch?v=LPZh9BOjkQs/>
- Willison, S. *Vibe Engineering y Agentic Engineering Patterns*. <https://simonwillison.net/2025/Oct/7/vibe-engineering/> · <https://simonwillison.net/guides/agentic-engineering-patterns/>
- Karpathy, A. *From Vibe Coding to Agentic Engineering*. <https://www.youtube.com/watch?v=96jN2OCOfLs>
- Osmani, A. *Agentic Engineering*. <https://addyosmani.com/blog/agentic-engineering/>
- MIT. *Missing Semester 2026: Agentic Coding*. <https://missing.csail.mit.edu/2026/agentic-coding/>
- Raschka, S. *The Components of a Coding Agent y Using Local Coding Agents*. <https://magazine.sebastianraschka.com/>
- *Agent Skills*, estándar abierto. <https://agentskills.io/> · Anthropic Academy, *Introduction to Agent Skills*. <https://anthropic.skilljar.com/introduction-to-agent-skills>
- Documentación de Pi (extensiones, compactación, sesiones, seguridad, modelos) y sus extensiones de ejemplo. <https://pi.dev/docs/latest/>
- Zechner, M. *The Pi coding agent y What if you don't need MCP?*. <https://mariozechner.at/>
- Copes, F. *A Deep Dive into Open-Weight AI Models*. <https://flaviocopes.com/open-weight-models/>
- Ollama <https://ollama.com/> · LiteLLM <https://github.com/BerriAI/litellm>
- CCAD-UNC: sitio <https://supercomputo.unc.edu.ar/ccad/>, wiki <https://wiki.ccad.unc.edu.ar/> y apertura de cuentas <https://wiki.ccad.unc.edu.ar/empezar/abrir-cuenta.html>

4. Justificación

En los últimos meses los agentes de codificación —programas que leen, escriben y ejecutan código de manera autónoma bajo la supervisión de una persona— pasaron a convertirse en la herramienta principal de trabajo en gran parte de la industria del software. En filadd (empresa donde trabajan Agustín y Diego) el código se escribe hoy de esa manera, y lo que se evalúa al incorporar a alguien ya no es solamente cuánto sabe programar, sino si es capaz de dirigir a un agente, revisar lo que produce y responder por el resultado. Los estudiantes avanzados y programadores Junior están enfrentando esta situación como una nueva exigencia.

A lo largo de seis encuentros semanales cada participante recorre la transición completa sobre un proyecto propio: primero experimenta los problemas que produce el uso sin estructura —código que funciona pero que no entiende y errores que se acumulan— y después incorpora, uno por uno, los hábitos que los resuelven: planificación, revisión de cambios, pruebas, documentación y registro de las decisiones. El impacto que buscamos, sin embargo, no es la destreza con una herramienta determinada —la que usamos en clase será distinta en dos años—, sino la idea que ordena todo lo demás: la responsabilidad por el software es de la persona y no de la IA. "Lo escribió el agente" no explica una vulnerabilidad, una decisión de arquitectura equivocada ni un sistema que nadie puede mantener.